SESSION LOG · POWEROPS_V1

Prompts & Tasks Log

Every prompt given during the build of PowerOps_v1, in order, paired with the work performed in response. Companion document to `PowerOps_Architecture.pdf`.

24	6	3	1
prompts	notebooks built	documents produced	app shipped

01

```
clear the previous prompt
```

Discarded an initial, interrupted paste of the full project specification before any work began.

02

[Full PowerOps_v1 specification — 15 numbered prompts plus a Final Master Prompt, covering architecture design, sample data, six build notebooks, the Streamlit UI, a src/ refactor, requirements/README, a management dashboard, conversational follow-ups, and a RAG failure-mode review]

Read the real dataset first rather than assuming the spec's generic schema; produced the requested architecture deliverable (data flow, project structure, packages, env vars, retrieval and escalation strategy, notebook sequence) adapted to the actual Jira-export fields; flagged concrete schema mismatches (no description field, no "Critical" priority, no literal "Open"/"Blocked" statuses) and asked four clarifying questions before writing any code.

03

[Answers to the four clarifying questions: OpenAI for embeddings + chat; literal-only priority matching, no "critical"→"Urgent" guessing; Summary + metadata as page content; build notebook-by-notebook with review pauses]

Scaffolded the project (data/notebooks/src) and a dedicated virtual environment (later rebuilt on Python 3.12 after Python 3.14 broke a Pinecone dependency); built and executed Notebook 01 against the real 1,000-issue dataset — validation, normalization, and distribution profiling.

04

```
Lets work on Notebook 02
```

Built and executed Notebook 02: designed the page_content template and Pinecone-safe metadata schema, empirically confirmed chunking was unnecessary for this dataset, and defined a stable idempotent vector-ID scheme. Created `.env.example` and `.gitignore`.

05

```
what is my project root folder
```

Answered the project root path. Discovered a real Pinecone API key had been pasted into the shareable `.env.example` template instead of the gitignored `.env`, flagged it as a security issue, and asked for it to be corrected.

06

```
ok moved and pine cone api key is added
```

Verified `PINECONE_API_KEY`; found `.env.example` had been deleted rather than copied and recreated it; found `OPENAI_API_KEY` missing entirely and asked for it to be added.

07

```
done
```

Rechecked `.env`: the `OPENAI_API_KEY` line existed but its value was blank. Asked again for the real key to be added.

08

```
done
```

Rechecked `.env`: `OPENAI_API_KEY` now had a value (75 characters). Installed the embedding/Pinecone packages and built and ran Notebook 03 in the background.

09

```
done
```

Notebook 03 failed with an OpenAI 401 error. Diagnosed that `OPENAI_API_KEY` held the exact same value as `PINECONE_API_KEY` — a copy-paste mix-up — and asked for it to be fixed. The recheck afterward showed the value still unchanged.

10

```
check now
```

Rechecked `.env`: the two keys had been swapped/partially cleared (Pinecone key now effectively empty, OpenAI key still carrying the Pinecone-style prefix). Asked for both lines to be set correctly, with the expected format for each.

11

```
fixed the keys
```

Verified both keys now had correct, distinct, properly-prefixed values. Re-ran Notebook 03, which failed this time with an OpenAI 429 "insufficient quota" error; asked for billing credits to be added.

12

```
re-run fixed the credit
```

Re-ran Notebook 03 successfully: embedded and upserted all 1,000 documents into a new serverless Pinecone index (`powerops-v1`), verified the vector count matched, and ran exploratory semantic queries.

13

```
Notebook 04
```

Built and executed Notebook 04: a fully deterministic query-filter parser driven by the real extracted vocabulary (issue-key regex, literal team/priority/status matching, two-tier assignee disambiguation), combined with Pinecone metadata filtering plus semantic search. Validated against 12 real questions, including deliberately unanswerable ones.

14

```
Notebook 05
```

Built and executed Notebook 05: implemented `ask_powerops()` with a grounding-focused system prompt and context construction. Testing surfaced a real gap — the escalation flag never fired even when the model's own answer said it found nothing.

15

```
Notebook 06
```

Built and executed Notebook 06: replaced the placeholder escalation logic with six deterministic evidence checks and persistent escalation records. Re-verified that all of Notebook 05's broken cases now escalate correctly — including one where the model's own framing was subtly misleading despite the correct escalation firing underneath.

16

```
[Answer to "what would you like to build next?"]: Streamlit UI (app.py)
```

Built `app.py`, wiring the full pipeline into an interactive UI. Installed Streamlit and a compatible Playwright/Chromium build (working around a macOS-version incompatibility), then launched the app and drove it end-to-end with a real headless browser — confirming both the grounded-answer path and the escalation path rendered correctly with zero console errors.

17

```
run this app so i can test it
```

Relaunched the Streamlit app in the background and shared the local URL along with suggested test questions covering both the grounded-answer and escalation paths.

18

```
Generate a Document which gives the into to this application, solution,
architecture with the diagram and flow, detailed break down of the
architecture, prompts, future to expand the system.
```

Authored a full architecture reference (overview, two process-flow diagrams, a section per notebook with real findings, the system prompts, design decisions, limitations, and roadmap), designed a dedicated visual identity for it, and published it as a shareable web artifact.

19

```
generate as a pdf document and place it in project root directory
```

Rebuilt the document as a standalone, print-ready HTML file; pre-rendered its Mermaid diagrams to static SVGs (the artifact viewer's live diagram rendering doesn't carry over to a static PDF export) and fixed several real rendering defects — illegible tiny diagram text, a mis-paginated blank page, clipped multi-line node labels — before exporting `PowerOps_Architecture.pdf` to the project root.

20

```
restart the app
```

Stopped and relaunched the Streamlit app, confirmed it was reachable again at `localhost:8501`.

21

```
give me the steps to push it to github
```

Checked the repository's git/GitHub CLI state (not yet a git repo; gh already authenticated) and provided copy-pasteable `git init` / commit / `gh repo create` steps, flagging project-specific notes (recommended private repo, what would get committed).

22

```
create a detailed architecture flow diagram for this application and list the points on the steps used to create this application , embed this information @PowerOps_Architecture.pdf under addendum section
```

Designed and embedded a new single, more detailed end-to-end architecture diagram (combining ingestion and query-time into one continuous flow) plus an 11-step "how this was built" log, under a new Addendum section. Regenerated the PDF (19 pages).

23

```
Include the system prompts used and regenerate the Architecture document
```

Added a third Addendum subsection reproducing both of PowerOps' actual LLM system prompts in full (the grounding prompt and the comparison-only structured-output parser prompt). Regenerated the PDF (21 pages).

24

```
export the list of prompts i have used here and tasks performed in a new prompt.pdf under the project directory
```

Compiled this session log and exported it as `prompt.pdf` at the project root.

PowerOps_v1 — prompts and tasks log, generated from the full build session.